Optimización

GIA — UPC

Práctica 2

Diseño de la red de sensorización para l'Eixample de Barcelona

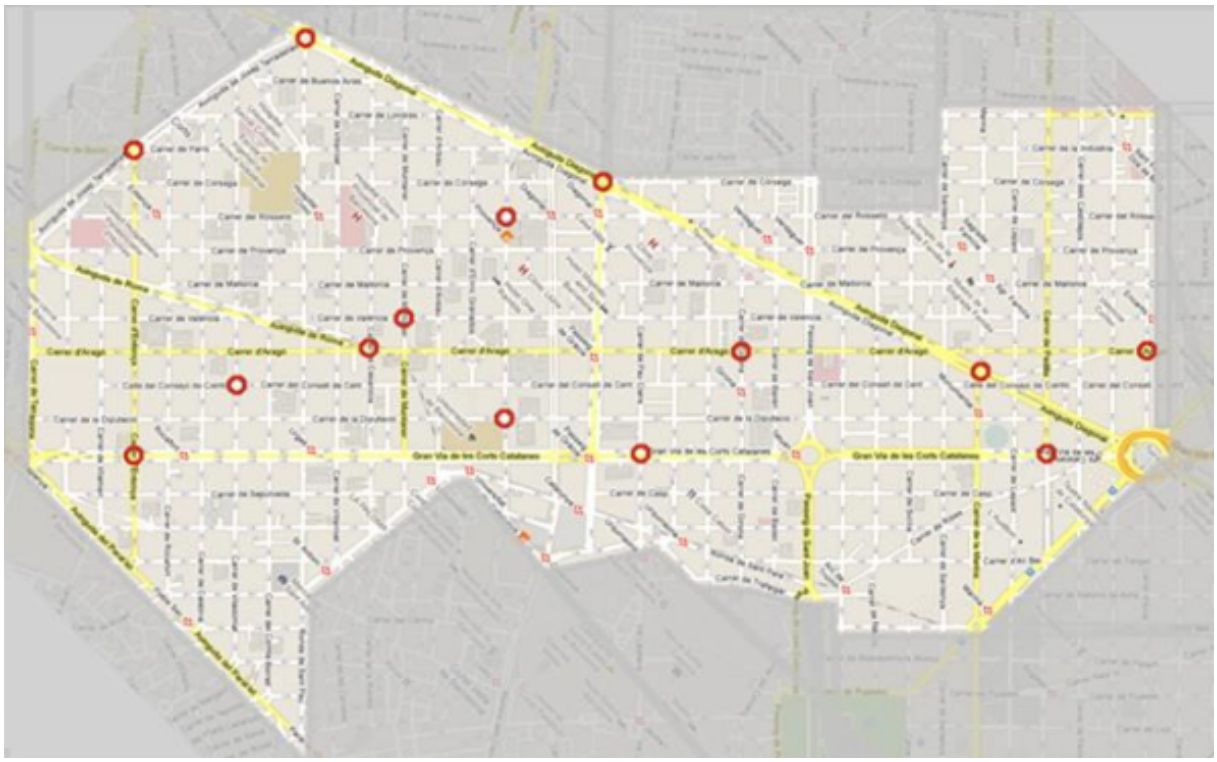

Figura 1: Red de intersecciones de l'Eixample de Barcelona.

Autores

Jan Estrada — DNI: `26314272H`
Adrian COMPLETAR — DNI: `COMPLETAR`

Fecha

26 de marzo de 2026

Índice

1. Introducción **3**

1.1. Datos del problema 3

2. Organización del proyecto **4**

3. Formulación matemática **4**

3.1. Apartado A — Modelo base 4

3.1.1. Conjuntos 4

3.1.2. Parámetros 5

3.1.3. Variables de decisión 5

3.1.4. Función objetivo 5

3.1.5. Restricciones 6

3.1.6. Justificación de la restricción de detección 6

3.1.7. Naturaleza del problema 6

3.2. Apartado B — Restricción de separación 7

3.2.1. Conjunto adicional 7

3.2.2. Restricción adicional 7

4. Preprocesado en Python **7**

4.1. Parser y generación del `.dat` 8

4.2. Verificación de conflictos entre nodos fijos 8

5. Implementación en AMPL **9**

5.1. Elección del solver 9

5.2. Fichero de datos unificado 9

5.3. Modelo del apartado A (`practica2_a.mod`) 10

5.4. Modelo del apartado B (`practica2_b.mod`) 11

5.5. Ficheros `.run` 11

6. Resultados **11**

6.1. Resumen general 12

6.2. Sensores seleccionados 12

6.2.1. Análisis comparativo de los sensores 13
6.3. Caminos detectados 13
6.4. Dominancia del camino 1513 14

7. Verificación de coherencia **14**

8. Conclusiones **15**

1. Introducción

El problema planteado en esta práctica consiste en diseñar la ubicación óptima de sensores de detección de vehículos en una red urbana de intersecciones del distrito de l'Eixample de Barcelona. El objetivo es maximizar el flujo total de vehículos captado por los sensores, sujeto a un conjunto de restricciones operativas.

En concreto, se pueden instalar como máximo 15 sensores y se debe decidir en qué intersecciones colocarlos. Algunas intersecciones vienen marcadas como *fijas* (el sensor debe colocarse obligatoriamente en ellas) y otras como *prohibidas* (no pueden albergar un sensor bajo ninguna circunstancia). Además, un camino (ruta de vehículos) solo se considera *detectado* si al menos dos de las intersecciones que recorre disponen de un sensor instalado.

Se distinguen dos variantes del problema:

- **Apartado A:** formulación base con el límite de 15 sensores, nodos fijos, nodos prohibidos y detección de caminos.
- **Apartado B:** se añade una restricción técnica adicional que impide que dos sensores se sitúen en intersecciones vecinas, es decir, separadas por una distancia menor o igual a 300 metros.

Para resolver ambas variantes se ha seguido un flujo de trabajo en cuatro fases: formulación matemática del modelo como un problema de programación lineal entera binaria (PLEB), preprocesado del fichero de datos original mediante scripts de Python, implementación en lenguaje AMPL y resolución con el solver Gurobi, y finalmente validación e interpretación de los resultados obtenidos.

1.1. Datos del problema

Los datos proporcionados por la asignatura incluyen:

- **457 intersecciones** que conforman la red urbana de l'Eixample.
- **42 caminos** (rutas de vehículos), cada uno con un flujo estrictamente positivo asociado.
- **8 intersecciones fijas** donde el sensor es obligatorio: 30, 78, 44628, 45173, 45481, 45555, 45787 y 49180.
- **3 intersecciones prohibidas** donde no puede haber sensor: 54977, 73703 y 68.
- **542 pares camino–intersección** que definen qué intersecciones forman parte de cada camino.

- **1284 pares de vecindad** (tras normalización) entre intersecciones a 300 m o menos, usados en el apartado B.

Un aspecto importante es que de los 15 sensores disponibles, 8 ya vienen fijados de antemano, por lo que el modelo solo decide libremente la ubicación de los 7 sensores restantes. Esto reduce significativamente el espacio de búsqueda, aunque el problema sigue siendo combinatorio.

2. Organización del proyecto

Para mantener una estructura clara y separar versiones intermedias de los archivos definitivos, el trabajo se organizó dentro de una carpeta principal `definitivo/`, subdividida en cuatro bloques funcionales:

- `ampl/`: contiene los modelos `.mod` (uno por apartado), el fichero de datos `practica2.dat` y los archivos `.run` que orquestan la ejecución.
- `tools/`: scripts de Python encargados de parsear el fichero crudo original y generar automáticamente el `.dat`, así como verificaciones de consistencia previas.
- `datos_crudos/`: copia íntegra del fichero original entregado por la asignatura, sin modificaciones.
- `notes/`: notas internas de redacción y decisiones consolidadas durante el desarrollo.

Esta organización permite separar con claridad el dato original del preprocesado y de la implementación final, y garantiza que el fichero `.dat` no se ha construido manualmente sino mediante un proceso reproducible.

3. Formulación matemática

A continuación se presenta la formulación completa de los dos modelos de programación lineal entera binaria. Se detalla primero el modelo base del apartado A y después la extensión del apartado B.

3.1. Apartado A — Modelo base

3.1.1. Conjuntos

- I: conjunto de todas las intersecciones de la red ($|I| = 457$).
- P: conjunto de todos los caminos ($|P| = 42$).

- $F \subseteq I$: conjunto de intersecciones fijas ($|F| = 8$), donde obligatoriamente debe instalarse un sensor.
- $Q \subseteq I$: conjunto de intersecciones prohibidas ($|Q| = 3$), donde no puede situarse ningún sensor.
- $I_p \subseteq I$: subconjunto de intersecciones que pertenecen al camino $p \in P$. Este conjunto se materializa en la implementación mediante la relación bidimensional `PATH_INTERSECTIONS` $\subseteq P \times I$, que contiene 542 pares.

3.1.2. Parámetros

- $f_p > 0$: flujo de vehículos asociado al camino $p \in P$. El enunciado garantiza que todos los flujos son estrictamente positivos, lo cual es clave para la correcta formulación del modelo (véase la justificación más adelante).

3.1.3. Variables de decisión

Se utilizan dos familias de variables binarias:

- $x_i \in \{0, 1\}$, $\forall\, i \in I$: vale 1 si se instala un sensor en la intersección i, y 0 en caso contrario.
- $y_p \in \{0, 1\}$, $\forall\, p \in P$: vale 1 si el camino p se considera detectado, y 0 en caso contrario.

La variable y_p es una **variable auxiliar**: no representa una decisión física (no se "instala" nada en un camino), sino que sirve para vincular la presencia de sensores en las intersecciones con el flujo que puede contabilizarse en la función objetivo.

3.1.4. Función objetivo

Se desea maximizar el flujo total detectado por los sensores:

$$\text{máx } Z = \sum_{p \in P} f_p \, y_p$$

3.1.5. Restricciones

$$\sum_{i \in I} x_i \leq 15 \qquad \text{(límite máximo de sensores)} \tag{1}$$

$$x_i = 1 \quad \forall\, i \in F \qquad \text{(intersecciones fijas)} \tag{2}$$

$$x_i = 0 \quad \forall\, i \in Q \qquad \text{(intersecciones prohibidas)} \tag{3}$$

$$\sum_{i \in I_p} x_i \geq 2\, y_p \quad \forall\, p \in P \qquad \text{(detección de caminos)} \tag{4}$$

3.1.6. Justificación de la restricción de detección

La restricción (4) constituye el núcleo de la modelización y merece una explicación detallada:

- **Si** $y_p = 1$ (el camino se marca como detectado), la restricción exige que $\sum_{i \in I_p} x_i \geq 2$, es decir, que al menos dos intersecciones del camino tengan sensor instalado. Esto concuerda exactamente con la definición de camino detectado del enunciado.
- **Si** $y_p = 0$ (el camino no se marca como detectado), la restricción se convierte en $\sum_{i \in I_p} x_i \geq 0$, que se satisface trivialmente para cualquier asignación de sensores.

Surge entonces la pregunta: ¿no falta una restricción que fuerce $y_p = 1$ cuando un camino *sí* tiene dos o más sensores? La respuesta es **no**, y la razón es importante: dado que todos los flujos f_p son estrictamente positivos y la función objetivo es de maximización, al solver siempre le interesa activar $y_p = 1$ cuando la restricción (4) lo permite, porque hacerlo incrementa el valor de Z. Si un camino tiene dos o más sensores y y_p pudiera valer 1, el solver lo pondrá a 1 por puro interés de maximización. Si no existiera esta garantía (por ejemplo, si algún flujo fuera negativo o nulo), sería necesario añadir una restricción adicional del tipo $y_p \leq \lfloor \sum_{i \in I_p} x_i / 2 \rfloor$, complicando innecesariamente el modelo.

3.1.7. Naturaleza del problema

El modelo resultante es un problema de **programación lineal entera binaria** (PLEB o Binary Integer Program, BIP): todas las variables son binarias y todas las restricciones y la función objetivo son lineales. Este tipo de problemas se resuelve mediante algoritmos de ramificación y acotación (branch and bound), que es exactamente lo que realizan solvers como Gurobi o CPLEX.

El tamaño del modelo es moderado: 457 variables x_i + 42 variables y_p = 499 variables binarias, con $1 + 8 + 3 + 42 = 54$ restricciones en el apartado A. Esto permite que los solvers modernos lo resuelvan en fracciones de segundo.

3.2. Apartado B — Restricción de separación

El apartado B reutiliza íntegramente la formulación del apartado A e incorpora una restricción técnica adicional: dos sensores no pueden situarse simultáneamente en intersecciones *vecinas*, definidas como aquellas separadas por una distancia menor o igual a 300 metros.

3.2.1. Conjunto adicional

- $N \subseteq I \times I$: conjunto de pares de intersecciones vecinas ($|N| = 1284$ pares normalizados).

En la implementación, cada par (i, j) se almacena de forma normalizada como $(\text{mín}(i, j), \text{máx}(i, j))$, lo que elimina duplicados simétricos. Sin esta normalización, el par (a, b) y el par (b, a) generarían dos restricciones idénticas en el modelo, lo cual no causa errores pero sí ineficiencia innecesaria.

3.2.2. Restricción adicional

$$x_i + x_j \leq 1 \quad \forall\, (i, j) \in N \tag{5}$$

Esta restricción impide que dos sensores queden a una distancia inferior a la mínima permitida. Con ella, el modelo del apartado B tiene $54 + 1284 = 1338$ restricciones, un incremento notable pero que Gurobi sigue resolviendo de forma casi instantánea.

Al añadir la restricción (5), el conjunto factible del apartado B se reduce respecto al del apartado A: toda solución factible de B también es factible en A, pero no al revés. Formalmente, $\mathcal{F}_B \subseteq \mathcal{F}_A$. Como consecuencia directa:

$$Z_B^* \leq Z_A^*$$

es decir, el flujo óptimo del apartado B no puede superar al del A. Si los valores difieren, la diferencia $Z_A^* - Z_B^*$ puede interpretarse como el **coste** de imponer la restricción de separación mínima entre sensores.

Antes de resolver el apartado B, consideramos que este coste podría ser significativo, dado que 1284 pares de vecindad sobre 457 intersecciones representan una red bastante densa. Sin embargo, como se verá en los resultados, la pérdida fue sorprendentemente pequeña.

4. Preprocesado en Python

El fichero original proporcionado por la asignatura (`OPT25-26_Datos práctica 2.txt`) no está en un formato directamente utilizable por AMPL. Por ello se desarrollaron dos

scripts auxiliares muy sencillos en Python cuyo único propósito es transformar y verificar los datos antes de ejecutar el solver.

4.1. Parser y generación del `.dat`

El script `tools/build_ampl_data.py` implementa un parser directo y minimalista:

1. Recorre el fichero crudo línea a línea, detectando cada sección por su cabecera de texto (`INTERSECTIONS`, `PATHS`, `FIXED`, `PROHIBITED`, `path_flow`, `path_intersections`, `intersection_neighborhood`).
2. Acumula las líneas de cada sección hasta encontrar la siguiente cabecera.
3. Genera el fichero `practica2.dat` con la sintaxis exacta que AMPL espera: conjuntos con `set ... :=`, parámetros con `param ... :=`, y conjuntos relacionales con tuplas en formato `(a, b)`.
4. Para el conjunto `NEIGHBORS`, normaliza cada par eliminando vecindades reflexivas (un nodo no es vecino de sí mismo) y duplicados simétricos: `(min(a,b), max(a,b))`.

Al terminar, el script imprime un resumen de cardinalidades que usamos como comprobación de coherencia:

Listing 1: Salida del parser

```
Intersections: 457
Paths: 42
Path intersections: 542
Neighbors: 1284
```

Se optó por un parser sencillo en lugar de uno más complejo con validaciones exhaustivas, ya que el formato del fichero crudo es regular y predecible. Cualquier inconsistencia en los datos (referencias a intersecciones inexistentes, flujos negativos, etc.) será detectada inmediatamente por AMPL durante la carga del `.dat`, gracias a las declaraciones `within` y `>0` del modelo.

4.2. Verificación de conflictos entre nodos fijos

El script `tools/verify_fixed_intersections.py` realiza una comprobación específica previa al apartado B: recorre la lista de nodos `FIXED` y la estructura de vecindad para detectar si existen dos nodos fijos que además sean vecinos entre sí.

¿Por qué es necesaria esta verificación? Si existieran dos nodos fijos vecinos, el modelo del apartado B sería **intrínsecamente infactible**:

- La restricción (2) obligaría a instalar sensor en ambos: $x_i = x_j = 1$.
- La restricción (5) lo prohibiría: $x_i + x_j \leq 1$.
- Ambas restricciones serían mutuamente excluyentes, haciendo el problema irresoluble.

Este tipo de infactibilidad es difícil de diagnosticar a posteriori a partir del mensaje del solver, por lo que conviene detectarla *antes* de lanzar la ejecución. La comprobación es rápida (unas pocas líneas de código) y ahorra potenciales dolores de cabeza.

La ejecución del script confirmó que **no existen conflictos** entre nodos fijos vecinos, por lo que procedimos con confianza a resolver el modelo del apartado B.

5. Implementación en AMPL

5.1. Elección del solver

Se utilizó **Gurobi 13.0.0** como solver para ambos apartados. Gurobi es un solver comercial de alto rendimiento para problemas de programación lineal, entera y mixta, ampliamente utilizado tanto en ámbitos académicos como industriales. Se eligió sobre otras alternativas (como CPLEX o los solvers open-source) por estar disponible localmente en la instalación de AMPL del equipo de trabajo y por ofrecer garantía de optimalidad global para problemas de este tipo.

Al tratarse de un problema de tamaño moderado (499 variables binarias, entre 54 y 1338 restricciones), cualquier solver MILP competente habría sido adecuado. De hecho, los tiempos de resolución fueron prácticamente instantáneos en ambos casos.

5.2. Fichero de datos unificado

Una decisión de diseño importante fue utilizar un **único fichero** `practica2.dat` para ambos apartados, en lugar de uno por modelo. Este fichero contiene todos los conjuntos y parámetros, incluido `NEIGHBORS`.

En el modelo del apartado A, el conjunto `NEIGHBORS` se declara pero no se utiliza en ninguna restricción:

```
# Declarado para compartir practica2.dat con el apartado B.
set NEIGHBORS within {INTERSECTIONS, INTERSECTIONS};
```

AMPL requiere que todo conjunto presente en el fichero `.dat` esté declarado en el `.mod`, aunque no participe en ninguna restricción. Esta declaración "vacía" es inocua y evita tener que mantener dos ficheros de datos distintos, lo cual sería propenso a errores de sincronización.

5.3. Modelo del apartado A (`practica2_a.mod`)

La implementación sigue fielmente la formulación matemática:

Listing 2: Modelo AMPL completo del apartado A

```
set INTERSECTIONS;
set PATHS;
set FIXED within INTERSECTIONS;
set PROHIBITED within INTERSECTIONS;
set PATH_INTERSECTIONS within {PATHS, INTERSECTIONS};
# Declarado para compartir practica2.dat con el apartado B.
set NEIGHBORS within {INTERSECTIONS, INTERSECTIONS};

param PATH_FLOW {PATHS} > 0;

var x {i in INTERSECTIONS} binary;
var y {p in PATHS} binary;

maximize total_flow:
    sum {p in PATHS} PATH_FLOW[p] * y[p];

subject to sensor_budget:
    sum {i in INTERSECTIONS} x[i] <= 15;

subject to detected_paths {p in PATHS}:
    sum {(p, i) in PATH_INTERSECTIONS} x[i] >= 2 * y[p];

subject to fixed_sensors {i in FIXED}:
    x[i] = 1;

subject to prohibited_sensors {i in PROHIBITED}:
    x[i] = 0;
```

Algunos aspectos relevantes de la implementación:

- La declaración `PATH_FLOW {PATHS} >0` impone positividad estricta del flujo. Si algún camino tuviera flujo nulo o negativo en el `.dat`, AMPL rechazaría los datos antes de resolver, evitando resultados incorrectos.
- La restricción `detected_paths` utiliza la notación `(p, i) in PATH_INTERSECTIONS` para iterar únicamente sobre los pares camino–intersección que realmente existen (542 pares), lo que es mucho más eficiente que un indexado completo sobre $P \times I$ (que serían $42 \times 457 = 19194$ combinaciones).
- Las declaraciones `within INTERSECTIONS` en `FIXED` y `PROHIBITED` actúan como

comprobación de integridad referencial: si el `.dat` contuviera un nodo fijo que no existiera en `INTERSECTIONS`, AMPL daría error inmediatamente.

5.4. Modelo del apartado B (`practica2_b.mod`)

El modelo del apartado B es idéntico al del apartado A, con la adición de una única familia de restricciones:

Listing 3: Restricción adicional en el apartado B

```
subject to separated_sensors {(i, j) in NEIGHBORS}:
    x[i] + x[j] <= 1;
```

Dado que el conjunto `NEIGHBORS` está normalizado (cada par aparece una sola vez como $(\text{mín}, \text{máx})$), no se generan restricciones duplicadas.

5.5. Ficheros `.run`

Los ficheros `.run` orquestan la ejecución completa: cargan el modelo y los datos, seleccionan el solver y muestran los resultados. Ambos tienen la misma estructura:

Listing 4: Fichero `practica2_a.run` (análogo para el apartado B)

```
model practica2_a.mod;
data practica2.dat;
option solver gurobi;
solve;
display total_flow;
display {i in INTERSECTIONS : x[i] = 1} x[i];
display {p in PATHS : y[p] = 1} y[p];
```

Se ha optado por mostrar únicamente las variables activas (con valor 1) mediante la condición `x[i] = 1` en el `display`. Esto produce una salida limpia y legible, en lugar de listar las 457 intersecciones con sus valores (casi todos 0).

6. Resultados

Ambos modelos se resolvieron localmente con **Gurobi 13.0.0** a través de la interfaz de AMPL. Los dos alcanzaron el estado `optimal solution`, lo que garantiza que las soluciones reportadas son **globalmente óptimas**: no existe ninguna otra asignación de sensores que produzca un flujo mayor.

6.1. Resumen general

Cuadro 1: Resumen comparativo de resultados

Magnitud	**Apartado A**	**Apartado B**
Estado del solver	Óptimo	Óptimo
Flujo total detectado	350,734	350,178
Número de sensores	15	15
Caminos detectados	24	24
Iteraciones símplex	43	75
Nodos de ramificación	1	1
Solver	Gurobi 13.0.0	Gurobi 13.0.0

El flujo total detectado del apartado A ($Z_A^* = 350{,}734$) es ligeramente superior al del apartado B ($Z_B^* = 350{,}178$), tal como se esperaba al tratarse de un conjunto factible más restrictivo. La diferencia es:

$$\Delta Z = Z_A^* - Z_B^* = 350{,}734 - 350{,}178 = 0{,}556$$

Este resultado fue sorprendente: dado que el apartado B añade 1284 restricciones de vecindad, esperábamos una pérdida de flujo más significativa. Sin embargo, la diferencia es de apenas un 0,16 % del total. Esto sugiere que la topología de l'Eixample es lo suficientemente amplia como para que los sensores puedan redistribuirse sin una gran penalización.

Otro dato notable es que ambos apartados requirieron **solo 1 nodo de ramificación** en el algoritmo branch-and-bound. Esto indica que la relajación lineal del problema (es decir, el mismo modelo pero con variables continuas en $[0, 1]$ en vez de binarias) produce una solución muy cercana a la entera, y Gurobi apenas necesita explorar el árbol de soluciones. Es un indicador de que el modelo está bien formulado y es "fácil" desde el punto de vista computacional.

6.2. Sensores seleccionados

Cuadro 2: Intersecciones con sensor en cada apartado (15 nodos)

Apartado	**Intersecciones con sensor** ($x_i = 1$)
A	5, 30, 78, 20349, 41633, 41970, 44494, 44609, 44628, 45173, 45481, 45555, 45787, 49180, 54839
B	5, 30, 78, 41633, 41653, 41970, 44522, 44609, 44628, 45173, 45481, 45555, 45787, 49180, 54839

6.2.1. Análisis comparativo de los sensores

Ambas soluciones comparten **13 de los 15 sensores**:

- Los **8 nodos fijos** (30, 78, 44628, 45173, 45481, 45555, 45787, 49180) aparecen en ambas soluciones, como imponen las restricciones.
- Los nodos **5, 41633, 41970, 44609** y **54839** se seleccionan libremente en ambos apartados. El hecho de que estos 5 nodos sean elegidos en las dos variantes indica que son intersecciones muy rentables: participan en muchos caminos de alto flujo y no entran en conflicto con ninguna restricción de vecindad.

Las diferencias se concentran en dos nodos:

- **Apartado A** incluye los nodos **20349** y **44494**, que no aparecen en B.
- **Apartado B** incluye los nodos **41653** y **44522** como sustitutos.

Estos cambios se deben a que la restricción de separación (5) invalida la combinación de nodos 20349 y/o 44494 con algún otro nodo seleccionado (por ser vecinos a menos de 300 m), forzando al solver a buscar las siguientes mejores alternativas. Es interesante notar que el solver no simplemente "elimina" dos nodos y deja huecos, sino que reemplaza por otros nodos que permiten seguir detectando un número igual de caminos (24 en ambos casos).

6.3. Caminos detectados

Cuadro 3: Caminos detectados ($y_p = 1$) en cada apartado

Apartado	**Caminos detectados**
A (24 caminos)	1439, 1441, 1451, 1479, 1483, 1484, 1488, 1492, 1493, 1499, 1500, 1504, 1513, 1516, 1518, 1536, 1537, 1541, 1547, 1549, 1550, 1560, 1561, 1577
B (24 caminos)	1445, 1451, 1479, 1483, 1484, 1488, 1492, 1493, 1499, 1500, 1504, 1513, 1516, 1518, 1525, 1536, 1537, 1541, 1547, 1549, 1550, 1560, 1561, 1577

Ambas soluciones detectan **24 de 42 caminos**, pero con ligeras diferencias en cuáles se detectan:

- El apartado A detecta los caminos **1439** ($f = 3{,}019$) y **1441** ($f = 0{,}202$), que no aparecen en B.

- El apartado B detecta los caminos **1445** ($f = 0{,}088$) y **1525** ($f = 2{,}577$), que no aparecen en A.

Esto es coherente con los cambios de sensores: al reubicar los nodos con sensor, cambian los caminos que cumplen la condición de tener al menos dos intersecciones sensorizadas. La diferencia neta de flujo es:

$$(3{,}019 + 0{,}202) - (0{,}088 + 2{,}577) = 3{,}221 - 2{,}665 = 0{,}556$$

que coincide exactamente con ΔZ calculado antes, lo cual sirve como comprobación cruzada de coherencia.

6.4. Dominancia del camino 1513

Un aspecto muy destacable es que el camino **1513** tiene un flujo de $f_{1513} = 261{,}109$ vehículos, que representa por sí solo aproximadamente el **74,5 %** del flujo total detectado. Este camino pasa por solo 6 intersecciones (44604, 44609, 44616, 44623, 44628, 44635), dos de las cuales (44609 y 44628) son seleccionadas como sensor en ambos apartados (44628 es un nodo fijo). Esto significa que la sola presencia de estos dos sensores justifica la mayor parte del valor objetivo.

El resto de los 23 caminos detectados aportan conjuntamente los $\approx 89{,}6$ unidades restantes, con contribuciones individuales mucho más modestas.

7. Verificación de coherencia

Para confirmar la validez de los resultados, se verificaron los siguientes criterios de coherencia sobre ambas soluciones:

- **Nodos fijos:** los 8 sensores fijos (30, 78, 44628, 45173, 45481, 45555, 45787, 49180) aparecen activados ($x_i = 1$) en ambas soluciones. ✓
- **Nodos prohibidos:** los 3 nodos prohibidos (54977, 73703, 68) no aparecen entre los sensores seleccionados en ningún apartado. ✓
- **Límite de sensores:** ambas soluciones instalan exactamente 15 sensores. ✓
- **Optimalidad:** Gurobi reporta `optimal solution` en ambos casos, con gap de MIP nulo en el apartado B (`absmipgap = 5.68e-14`, esencialmente cero). ✓
- **Relación $Z_B^* \leq Z_A^*$:** se cumple ($350{,}178 \leq 350{,}734$), confirmando la relación teórica entre los conjuntos factibles. ✓

- **Coherencia cruzada de flujos:** la diferencia $\Delta Z = 0{,}556$ coincide exactamente con la suma de flujos de los caminos ganados y perdidos entre A y B, como se calculó anteriormente. ✓
- **Separación en apartado B:** se comprobó visualmente que ningún par de nodos seleccionados en B aparece en el conjunto `NEIGHBORS`. ✓

8. Conclusiones

La resolución de esta práctica se ha abordado mediante una formulación de programación lineal entera binaria clara y compacta, implementada de forma directa en AMPL y resuelta con Gurobi 13.0.0. El preprocesado previo en Python, aunque sencillo e intencionadamente minimalista, ha sido útil para:

1. Evitar errores de transcripción manual en un fichero de datos con 457 intersecciones, 42 caminos, 542 pares camino–intersección y 1284 relaciones de vecindad.
2. Verificar la consistencia del dataset antes de ejecutar el solver, descartando infactibilidades estructurales (conflictos entre nodos fijos vecinos en el apartado B).
3. Generar un fichero `.dat` unificado, reutilizable por ambos modelos sin duplicar información.

Los resultados obtenidos permiten extraer las siguientes conclusiones:

- Ambos modelos son factibles y alcanzan el óptimo global, con tiempos de resolución prácticamente instantáneos.
- La restricción de separación del apartado B tiene un impacto operativo muy reducido ($\Delta Z \approx 0{,}16\,\%$). Esto sugiere que la topología de l'Eixample, con su estructura reticular regular, permite distribuir los sensores eficientemente incluso respetando la distancia mínima de 300 m.
- La solución está fuertemente dominada por el camino 1513, cuyo flujo (261,109) representa el 74,5 % del total detectado. Esto implica que la decisión más crítica es asegurar que este camino quede detectado, lo cual ocurre en ambos apartados.
- El hecho de que ambas soluciones utilicen los 15 sensores indica que la restricción es activa: con más sensores disponibles, podrían detectarse caminos adicionales.
- Los 5 nodos elegidos libremente que coinciden en ambos apartados (5, 41633, 41970, 44609, 54839) representan intersecciones de importancia estratégica en la red, al participar en múltiples caminos de alto flujo.

En una aplicación real sería imprescindible asegurar que las restricciones técnicas se cumplen exactamente como se han modelado, especialmente la distancia mínima entre sensores. Además, cabría estudiar la sensibilidad del modelo a variaciones en los flujos o en el número máximo de sensores, para evaluar la robustez de la solución ante cambios en las condiciones operativas.